

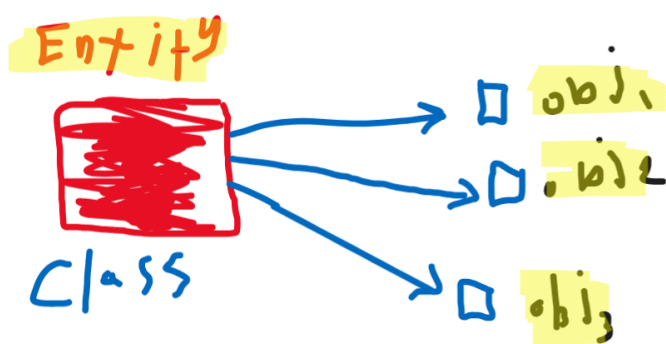
Introduction to Object-Oriented Programming

📝 Notes by Menna Reda

تخيل كد انا بعمل برنامج البرنامج دا بيحسب مجموع درجات الطلبة بياخد اسم وبيانات الطالب و ييدا يحسب درجاته المفروض لو انا هعمل حاجه زي كد المفروض هيدا اعمل متغير شايل اسم الطالب متغير شايل عمره و السنه الدراسيه و درجاته وايدا اجمع درجات المواد وأرجعه ناتج تخيل بق عندي طالب كمان مفيش مشكله هيدا اعرف المتغيرات دي كلها ثاني واحسب وارجع مجموع الدرجات تخيل بق عندي 100 طالب ماشي يسيدي هعمل array طب لحظه كد اراي لاي ولا لاي وغيره بق من مشاكل هربط كل طالب ببياناته تحديدا ازاي من هنا ظهرت فكرت ال OOP

البرمجه الكائنيه يعني انا بعمل كيان للطالب بعرف فيه المتغيرات المطلوبه وكل م احتاج طالب جديد هاخذ نسخه من الكيان دا مستقله بذاتها وبياناتها من غير اي لغبطه او سوء اداره للميموري و تنظيم تام حرفيا

اذن انا عندي كيان "Class" بخرن فيها متغيرات و ميثودز وبعمل منها نسخ "Objects" زي منا عايز



يبق كد عندي كود واحد اقدر
اعمل منه نسخ زي منا عايز

1. الكيان (ال Class)

```
public class Student
{
    // البيانات (Data / Fields / Properties)
    public string Name { get; set; }
    public int Age { get; set; }
    public string GradeLevel { get; set; }
    public List<double> Grades { get; set; } = new List<double>();

    // العمليات (Methods)
    public double CalculateTotalGrades()
    {
        double total = 0;
        foreach (var grade in Grades)
        {
            total += grade;
        }
        return total;
    }
}
```

2. الاستخدام في ال Main (تجنب العشوائية وتنظيم الميموري)

```
static void Main(string[] args)
{
    // إنشاء الطالب الأول (Object 1)
    Student student1 = new Student();
    student1.Name = "Menna";
}
```

```

student1.Age = 20;
student1.Grades.AddRange(new double[] { 95, 98, 100 });

// مستقل تماماً في الذاكرة - (Object 2) إنشاء الطالب الثاني
Student student2 = new Student();
student2.Name = "Ahmed";
student2.Age = 15;
student2.Grades.AddRange(new double[] { 80, 75, 90 });

// مفيش اكواد كثير عالفاضي (DRY تطبيق الـ) طباعة النتائج باستدعاء الميثود الموحدة
Console.WriteLine($"{student1.Name}'s Total: {student1.CalculateTotalGrades()}");
Console.WriteLine($"{student2.Name}'s Total: {student2.CalculateTotalGrades()}");
}

```

OOP stands for Object-Oriented Programming.

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.